

P08 Payment methods, discount reasons and No Charge

Model: Opus, high effort.

Branch: none. Commit directly to `main`.

Depends on: P07.

1. What to build, in one sentence

Add a test-only clock, then build configurable payment methods with frozen payment details and method corrections, fixed discount reasons with who funded them, and No Charge orders, exactly as P07 specified them.

2. Module

M10 Payments and the No Charge part of M16.

3. The spec is already written

P07 wrote the design into `docs/API-CONTRACT.md` sections "M10 Payments" and "M16 Settlement and Day Close", and `docs/DB-SCHEMA.md` sections 20 onwards.

Build exactly what they say. If the spec and this prompt disagree, the spec wins, and you tell me in your summary.

If the spec is silent on something you need, stop and ask rather than invent it.

4. Step 0. Save this prompt and check the repo

1. Save this entire prompt, exactly as given, to `docs/prompts/P08-payment-methods-discounts-no-charge.md`.
2. `git pull` . `git status` should show only that file.
3. `docs/prompts/README.md` must show P07 as Done. If not, stop.
4. Run `npm test` and record the count. It should match P06's "after" count, since P07 changed no code. If anything fails before you start, stop and tell me.

5. Files to read first

1. The M10 section, and the No Charge part of the M16 section, of `docs/API-CONTRACT.md`.
2. `docs/DB-SCHEMA.md` section 20 `paymentmethods` , and the new fields in sections 9, 12 and 17.
3. `docs/TEST-DATA.md` , all of it.
4. `server/utils/time.js` .
5. `server/models/Bill.js` , `server/models/Order.js` , `server/models/index.js` .
6. `server/services/billService.js` , `server/services/billPermissionService.js` ,

- `server/services/settingsService.js` .
 - 7. `server/validators/billValidators.js` ,
`server/validators/orderValidators.js` ,
`server/validators/common.js` and how P04 built
`reasonFields` .
 - 8. `server/scripts/provisionRestaurant.js` and
`server/scripts/seedDemo.js` .
 - 9. `server/config/cancelReasons.js` and its client
mirror from P04, as the pattern for new reason lists.
 - 10. On the client: the bill screen's payment and discount
panels, `ReasonPicker.jsx` , and the order screen.
-

6. Part A. A clock tests can set

The problem.

The golden day in `docs/TEST-DATA.md` happens at fixed times on 26 September 2026, including a bill paid at 12:02 AM that still belongs to the 26th.

Tests cannot reproduce that today, because services read the real time.

The change.

1. In `server/utils/time.js` , make `nowUtc()` read from a replaceable clock.
2. Add `setClockForTests(fnOrDate)` and `resetClockForTests()` . Both throw unless `NODE_ENV` is `test` .
3. In `server/services/` and `server/controllers/` , replace every `new Date()` that records when a

business event happened with `nowUtc()` . That includes at least: line `addedAt` , order `openedAt` , KOT `firedAt` and `readyAt` , stock movement times, and any `new Date()` in billing.

4. Leave the token service alone. Login and refresh times must stay real, or tokens break in tests.
5. Add a test that reads every file in `server/services/` and `server/controllers/` and fails on `new Date()` with no argument, except in `tokenService.js` . Same idea as P01's time display guard.

Every existing test must still pass with the clock at its default.

Commit on its own: `add a test clock` .

7. Part B. Payment methods

Build, exactly as specified:

1. The `PaymentMethod` model, registered in `server/models/index.js` .
2. `server/services/paymentMethodService.js` with `ensureDefaultPaymentMethods(restaurantId, { session })` , which creates `CASH` , `CARD` , `UPI` and the inactive `OTHER` by code if missing, and never changes one that exists. Use an upsert with `$setOnInsert` per code.
3. Call it from provisioning, inside its transaction, and lazily before listing methods or taking a payment, so

restaurants created before this prompt get their defaults without a migration.

4. The three endpoints, with validation and permissions from the spec.
5. Payment taking: remove the fixed enum from `payments[].method`, check the four rules in the service, and freeze `methodName`, `methodKind`, `tallyLedgerCode`, `commissionBps` and `businessDate` on each payment.
6. The payment correction endpoint, with its `corrections` history and the `PAYMENT_METHOD_CORRECTED` audit line. The closed-day refusal arrives in P10. Leave a clear comment at the point where P10 will add it.

Append the new audit actions and entity types this prompt writes, and the new error codes, to their lists. Append only.

8. Part C. Discount reasons

Build, exactly as specified:

1. `server/config/discountReasons.js`, and its client mirror with no imports, and a test that they match.
2. The discount request changes from `reason` to `reasonCode` plus `note`, with `fundedBy`. The old `reason` field is refused with 400, the way P04 did for cancels. Reuse P04's `reasonFields` helper.
3. The bill's `discount` gains `reasonCode` and `fundedBy`. Its existing `reason` text holds the note.

4. The new settings group `discounts` , with `cashierMayApplyPlatformDiscounts` , default `false`, audited like every setting.
5. Permission: when the setting is on, a CASHIER may apply a discount with a platform reason code and no other. Put the rule in `billPermissionService.js` , where every bill permission already lives, and move `POST /bills/:id/discount` off the plain `managers` role list so the service decides.
6. `DISCOUNT_APPLIED` audit lines gain `reasonCode` and `fundedBy` in `details` .

Update `scripts/seedDemo.js` and every existing test that sends a discount `reason` , to send `reasonCode` with a note instead, as P04 did for cancels.

9. Part D. No Charge

Build, exactly as specified:

1. `server/config/noChargeReasons.js` , its client mirror, and the mirror test.
2. `NO_CHARGE` appended to the order statuses, and the `noCharge` sub-object on orders.
3. `POST /orders/:orderId/no-charge` , with its four rules, in one transaction, freeing the table and writing `NO_CHARGE_GIVEN` .
4. Check every place in the server that lists or counts orders by status, so a `NO_CHARGE` order is never shown

as open, never billable, and never counted as a sale.
Search for the order status constants to find them.

10. The client

1. **Paying a bill.** The payment panel shows one large button per method the bill may use, from `GET /payment-methods`, in display order. On a Swiggy delivery bill, only Swiggy shows. The server still enforces the rules. Split payments keep working as today.
2. **Correcting a method.** On a paid bill, a MANAGER or OWNER sees "Change payment method" on each payment, with the method buttons and a required reason.
3. **Discounts.** The discount panel uses `ReasonPicker` with the discount reasons. For a platform reason, a small "Paid for by: Restaurant or Platform" choice appears. A cashier sees the discount panel only when the setting allows, and then only platform reasons.
4. **No Charge.** On the order screen, a MANAGER or OWNER sees "No Charge" in the order's menu. It shows the value at menu price, the reason buttons, and a confirm button.
5. **Settings.** A "Payment methods" section for the OWNER: list, add, edit name, order types, Tally code, commission, display order, active. Code and kind are shown but not editable after creation. And the cashier platform discount switch, in a "Discounts" section.

Follow `docs/DESIGN-SYSTEM.md` .

11. Tests

Clock

1. With the clock set to 26 Sep 2026 11:55 PM India time, a bill has business date 2026-09-26. Set to 27 Sep 12:02 AM, a payment on it has business date 2026-09-26.
2. The `new Date()` guard from Part A.

Payment methods

3. A new restaurant has exactly the four built-in methods. A restaurant from before this prompt gets them on first use. Running the ensure twice changes nothing, and never overwrites a renamed method.
4. Every rule in the spec's "taking a payment" section, with its error code: an inactive method, a method not allowed for the order type, a non-Swiggy method on a Swiggy bill, a Swiggy method on a dine-in bill.
5. A payment freezes name, kind, Tally code, commission and business date. Renaming the method afterwards does not change the payment.
6. A commission change affects only later payments.
7. Correcting a payment's method keeps the amount, records the history, writes the audit line, and refuses a method the bill may not use. A cashier may not correct.
8. Every existing payment test still passes, with `CASH` , `CARD` and `UPI` codes.

Discounts

9. Each reason code accepted. `OTHER` without a note refused. The old `reason` field refused.
10. `fundedBy: PLATFORM` with a non-platform reason refused.
11. With the setting off, a cashier is refused every discount. With it on, a cashier may apply `ZOMATO_GOLD` and is refused `REGULAR_GUEST`.
12. The client and server reason files match.

No Charge

13. Each of the four rules refuses with its message.
14. A valid No Charge frees the table, stores `noCharge` with `valueInPaise` equal to the live line totals, writes `NO_CHARGE_GIVEN`, and creates no bill and uses no invoice number.
15. The golden day's N01: College Sandwich on Table 29, reason `CORPORATE_OFFICE`, gives `valueInPaise` 23000, and the next bill takes the next invoice number with no gap.
16. A No Charge order never appears in the open orders list, the billing list, or any existing M6 sales figure.

Golden day pieces, with the clock set to the bill times in `docs/TEST-DATA.md` section 2

17. B05 is paid with Cash 50000 and UPI 29500, and both payments carry business date 2026-09-26.
18. B02 is discounted 7307 with `ZOMATO_GOLD` and paid by `ZOMATO_GOLD` 144600.
19. B14 is billed at 11:55 PM and paid by `ZOMATO_GOLD` 55200 at 12:02 AM on 27 September, and the payment's business date is 2026-09-26.

Create the golden day's payment methods in these tests with the exact codes the golden day uses: ZOMATO_GOLD , DINEOUT , EAZYDINER , ZOMATO , SWIGGY , with ZOMATO and SWIGGY linked to their platform codes.

Run the full suite at the end. Every test that passed before must still pass.

12. Non-negotiable rules that apply

"Store all money as whole paise integers."

"Never hard delete a bill, order, or stock entry." A corrected payment keeps its history. A No Charge order is kept.

"Bill numbers are generated on the server, are sequential, and are never reused." No Charge takes none.

"Check permissions on the server for every endpoint."

"Which business day a moment belongs to is decided only by businessDateFor ." The payment's businessDate uses it.

"Schema changes are additive."

13. Checks and golden day

C3 and C4 still hold on every bill, because no payment amount logic changes.

C7: a No Charge order never has a bill. Test 14 proves it.

Tests 15, 17, 18 and 19 reproduce golden day pieces at their real times.

14. Docs to update

1. `docs/PROJECT-STATE.md` :
 1. The date line.
 2. "Current stage": "Next: P09, On Hold accounts and platform payouts."
 3. Decision log, dated today: "Services read the time only through `nowUtc()` , which tests can set with `setClockForTests` . Token times stay real. | The golden day happens at fixed times, including a payment after midnight."
 4. Every detail where the build differed from the P07 spec, as a decision row with its reason, and the spec updated to match.
 5. "What changed recently": a P08 entry at the top, and the oldest moved to the archive.
 2. `docs/prompts/README.md` : mark P08 as Done.
-

15. Out of scope

On Hold accounts and payouts. That is P09.
The cash drawer, Day Close and the day lock. That is P10.
Any report.
UPI QR codes.

16. Done when

1. `npm test` passes, with before and after counts recorded.
2. `npm run lint` and `npm run build` pass.
3. By hand: add Zomato Gold as a platform method, bill a dine-in table, apply a Zomato Gold discount, pay with Zomato Gold, then correct it to UPI as a manager.
4. By hand: give an order No Charge and see the table free up with no bill.
5. Every doc in section 14 is updated.
6. Commits on `main`, one line each, for example:
 - `add a test clock`
 - `add payment methods`
 - `freeze payment details and add method`
 - `corrections`
 - `add discount reasons and funding`
 - `add no charge orders`
 - `update docs for p08`
7. Push `main`.
8. Print a short summary: commits, files changed, test counts before and after, and every place the build differed from the spec.